東南大學

数据库课程实验报告

软 件 学 院 院（系）软 件 工 程 专 业

学 号 71122135

学生姓名 谢鹏宇

目 录

第一章　第一部分

1.1 第一题

1.1.1 题目

按字母顺序列出所有艺术家类型（artist type）。

1.1.2 sql 语句及分析

SQL 语句：

```
SELECT name FROM artist_type ORDER BY name;
```

从 `artist_type` 表中提取所有艺术家类型的名称（`name` 字段），并按字母顺序升序排列。查询中使用的 `SELECT name` 表示仅获取艺术家类型的名称，而不包括其他字段如 `id`。`ORDER BY name` 语句指定了对查询结果按照 `name` 字段进行排序，确保返回的名称按字母顺序排列。执行该查询时，数据库会扫描 `artist_type` 表中的所有记录，提取 `name` 字段的值，并按字母升序排列这些名称。最终，查询结果将返回一个按字母顺序排列的艺术家类型名称列表，体现了 `artist_type` 表中所有艺术家类型名称的升序排列。

1.1.3 查询结果截图

	name
1	Character
2	Choir
3	Group
4	Orchestra
5	Other
6	Person

Figure 1.1 第一题查询结果

1.2 第二题

1.2.1 题目

查找披头士乐队（The Beatles）解散前在英国（United Kingdoms）发行的所有 12” Vinyl 格式的专辑。

具体要求：列出专辑名称和发行年份。对于同名的多个专辑，选择发行年份最早的专辑。先按发行年份然后按专辑名称对结果进行排序。提示：12” Vinyl 是一种媒体格式（medium format）。

1.2.2 sql 语句及分析

sql 语句：

```
select release.name as RELEASE_NSME, min(ri.date_year) as RELEASE_YEAR
from artist
join area on area.id = artist.area
join artist_credit_name as acn on artist.id = acn.artist
join release on release.artist_credit = acn.artist_credit
join medium on medium.release = release.id
join medium_format as mf on mf.id = medium.format
join release_info as ri on ri.release = release.id
where artist.name = 'The Beatles'
and ri.area = area.id
and  area.name = 'United Kingdom'
and mf.name = '12" Vinyl'
and (ri.date_year < artist.end_date_year
 or ri.date_year = artist.end_date_year and ri.date_month < artist.end_date_month
 or ri.date_year = artist.end_date_year and ri.date_month = artist.end_date_month
    and ri.date_day < artist.end_date_day)
and ri.date_year is not null
group by release.name
order by ri.date_year asc, release.name asc;
```

查询首先从多个表格中获取相关数据，这些表格通过适当的连接条件进行关联，接下来，查询使用 ‘WHERE‘ 子句进行多个过滤操作。首先，过滤出艺术家名称为“The Beatles”的记录。然后，筛选出地区为“United Kingdom”且专辑格式为“12” Vinyl”的专辑。此外，查询还检查专辑的发布日期是否早于乐队的结束日期。结束日期通过 ‘artist.end_date_year‘、‘artist.end_date_month‘ 和 ‘artist.end_date_day‘ 来表示。查询还确保发布日期字段（‘ri.date_year‘）不为空。

查询的结果字段包括专辑名称和该专辑的最早发布日期年份。通过 ‘GROUP BY‘ 子句，按专辑名称对结果进行分组，确保每个专辑仅返回一条记录。然后，查询按发布日期年份升序排序，并在同年内按专辑名称升序排列。最终，查询结果限制为前 10 条记录。

1.2.3 查询结果截图

	RELEASE_NSME	RELEASE_YEAR
1	Please Please Me	1963
2	With The Beatles	1963
3	A Hard Day's Night	1964
4	Beatles for Sale	1964
5	Help!	1965
6	Rubber Soul	1965
7	A Collection of Beatles Oldies	1966
8	Revolver	1966
9	Magical Mystery Tour	1967
10	Sgt. Pepper's Lonely Hearts Club Band	1967
11	The Beatles' First	1967
12	The Beatles	1968
13	Abbey Road	1969

Figure 1.2 第二题查询结果

1.3 第三题

1.3.1 题目

查找媒体格式为 Cassette 的十个最新专辑。

具体要求：打印专辑名称、艺术家姓名和发行年份。按发行年份、月份和日期从最新到最旧对输出进行排序，然后按专辑名称、艺术家姓名的字母顺序进行排序。

1.3.2 sql 语句及分析

sql 语句：

```
select r.name as RELEASE_NAME, a.name as ARTIST_NAME, ri.date_year as RELEASE_YEAR
from release as r
    join release_info as ri on ri.release = r.id
    join medium as m on m.release = r.id
    join artist_credit_name as acn on r.artist_credit = acn.artist_credit
    join artist as a on a.id = acn.artist
    join medium_format as mf on mf.id = m.format
where mf.name = 'Cassette'
order by ri.date_year desc, ri.date_month desc, ri.date_day desc, r.name asc, a.name asc
limit 10;
```

首先，查询通过连接 ‘release‘、‘release_info‘、‘medium‘、‘artist_credit_name‘ 和 ‘artist‘ 等多个表来筛选出媒体格式为 Cassette 的专辑。接着，通过 ‘ORDER BY‘ 子句按照发行年份、月份和日期降序排列，确保显示的是最新的专辑。在同一日期内，专辑按照名称和艺术家姓名的字母顺序升序排列。最终，使用 ‘LIMIT 10‘ 限制结果集仅包含最新的十张专辑。

1.3.3 查询结果截图

	RELEASE_NAME	ARTIST_NAME	RELEASE_YEAR
1	Lesions of a Different Kind	Undeath	2020
2	Heaven & Hell	Ava Max	2020
3	Termination Shock	Traveler	2020
4	Women in Music, Pt. III	HAIM	2020
5	Flashback	Haunt	2020
6	Moonth	Merce Lemon	2020
7	PUNANI	6ix9ine	2020
8	Deep Dyed	Deep Dyed	2020
9	Meurtrières	Meurtrières	2020
10	Moondog Vol.1	Dan Peters	2020

Figure 1.3 第三题查询结果

1.4 第四题

1.4.1 题目

查找每种类型中注释（comment）最长的作品。

具体要求：列出每种作品类型 (work type) 中注释 (comment) 最长的作品。如果存在并列，则应返回注释最长的所有作品。最终输出按作品类型升序排列，然后按作品名称字母顺序排列。排除注释为空（长度为 0）的作品。

1.4.2 sql 语句及分析

sql 语句：

```
WITH MaxCommentLength AS (
    SELECT
        w.type,
        MAX(LENGTH(w.comment)) AS max_comment_length
    FROM
        work AS w
    WHERE
        LENGTH(w.comment) > 0
    GROUP BY
        w.type
)
SELECT
```

```
    wt.name AS WORK_TYPE,
    w.name AS WORK_NAME,
    LENGTH(w.comment) AS COMMENT_LENGTH,
    w.comment AS COMMENT
FROM
    work AS w
JOIN
    work_type AS wt ON w.type = wt.id
JOIN
    MaxCommentLength AS mcl ON w.type = mcl.type
WHERE
    LENGTH(w.comment) = mcl.max_comment_length
ORDER BY
    wt.name ASC, w.name ASC;
```

首先通过一个名为 ‘MaxCommentLength‘ 的子查询来计算每种作品类型（‘work.type‘）下评论的最大长度。子查询中的 ‘MAX(LENGTH(w.comment))‘ 获取每种类型的最大评论长度，并且通过 ‘GROUP BY w.type‘ 按作品类型分组，只考虑评论长度大于零的记录。

接着，主查询从 ‘work‘ 表中获取所有作品的详细信息，并通过与 ‘work_type‘ 表的连接来获取作品类型的名称。查询还通过与 ‘MaxCommentLength‘ 子查询的连接，确保只返回评论长度等于该类型最大评论长度的作品。查询筛选出符合条件的作品，显示作品类型、作品名称、评论长度以及评论内容。

最终，查询结果按照作品类型名称（‘wt.name‘）和作品名称（‘w.name‘）升序排列，确保输出的作品列表按类型和名称顺序组织。

1.4.3 查询结果截图

	WORK_TYPE	WORK_NAME	COMMENT_LENGTH	COMMENT
1	Aria	Phi-Phi: La gamine charmante	92	arrangement by Jean Poiret and Dominique Tirmont for the show "Il était une fois …
2	Audio drama	The Adventures of Tom Bombadil	104	Brian Sibley drama for BBC Radio. Based not on the poem, but on material from The …
3	Ballet	Les Mariés de la tour Eiffel	85	Ballet in One Act by members of the Groupe des Six from the spectacle by Jean …
4	Cantata	Jauchzet dem Herrn, alle Welt TWV 7:20	89	One of the 5 versions by Telemann: Cantata for Bass, Trumpet, Oboe, strings and …
5	Concerto	Concerto in C major, RV 558, "con molti stromenti"	108	2 violins in tromba marina, 2 recorders, 2 trumpets, 2 mandolines, 2 chalumeaux, 2 …
6	Incidental music	Rabelais	57	musique de scène pour le spectacle de Jean-Louis Barrault
7	Madrigal	As Fair as Morn, as Fresh as May	49	The Second Set of Madrigals for 3-6 voices, no. 5
8	Mass	Officium Defunctorum (1605)	104	With chants researched and edited from 16th- and 17th-century Spanish sources by …
9	Motet	Leçons de ténèbres du vendredy saint: Première leçon	63	De lamentatione Jeremiae prophetae. Heth. Misericordiae Domini.
10	Musical	John Paul Jones	72	A musical. Not to be confused with the orchestral works derived from it.
11	Opera	Prince Igor	76	opera by Borodin, most contemporary performances shouldn't link to this work
12	Operetta	I Have Been in Love Before	53	English version of "Gern hab' ich die Frau'n geküsst"
13	Oratorio	Markus-Passion, BWV 247	54	Reconstructed by Dietmar Hellmann and Andreas Glöckner
14	Overture	Preludio in si bemolle minore	140	1910, Ed. F. Bongiovanni - sopra un Corale di Bach "Ho sperato in te, o signore" ("In di…
15	Partita	Embroidery digitizing	143	Embroidery digitizing | Flat rate digitizing | Vector redraw | free embroidery designs …
16	Play	Belicha	59	From a document in the British Library in London : Ms.29987
17	Poem	Ночное	109	«Если б мои не болели мозги, я бы заснуть не прочь. Рад, что в окошке не видно …
18	Poem	О собаках	109	«Не могу я видеть без грусти ежедневных собачьих драк, в этом маленьком …
19	Prose	The Man and the Birds	72	a Christmas story most notably narrated by radio broadcaster Paul Harvey
20	Quartet	The Irish Session Suite: Four Reels (4th Movement)	72	Paddy Fahy's Reel / Beare Island Reel / Dowd's Favourite / Boys of Malin
21	Sonata	Sonata for Wind Quartet no. 6 in F major	74	arranged for Wind Quartet by F. Berr from an Andante e Tema con variazioni
22	Song	Ragam Thanam Pallavi	180	Raga: Ragamalika(Ranjani, Manoranjani, Janaranjani, Sriranjani), Tala: Tisra Jampa (4 …
23	Song-cycle	Sonnets From the Portuguese	93	composed by Libby Larsen, set to Elizabeth Barrett Browning's ca. 1845 work of the …

Figure 1.4 第四题查询结果

24	Soundtrack	Empath Finale	115	contains "Theme From Star Trek (TV Series)" by Alexander Courage & Gene ...
25	Suite	Flatpyramid	180	We offer an extensive 3D model library that includes thousands of products spread ...
26	Symphonic poem	Kullervo, op. 7	53	for mezzo-soprano, baritone, male choir and orchestra
27	Symphony	Sinfonia concertante no. 11 in A major (KaiS Sc11 ICS 33)	147	Simphonie Concertante a plusieurs instruments; Sinfonia Concertante à Violino, ...
28	Zarzuela	Marina	17	original zarzuela
29	Zarzuela	Mirentxu	17	original zarzuela
30	Étude	Go From My Window	49	Fitzwilliam virginal book, nos. 9 (I) and 42 (II)

Figure 1.5 第四题查询结果（补）

1.5 第五题

1.5.1 题目

查找艺术家发行量最多的月份。

具体要求：对于每个艺术家类型（artist type）为 Person 且名字以 Elvis 开头的艺术家，找出其发行数量最多的月份。如果数量相同，则选择较早的月份。最终输出按发行量降序排列，然后按艺术家姓名字母顺序排列。排除未指定月份的发行。

1.5.2 sql 语句及分析

sql 语句：

```
SELECT
    fa.artist_name AS ARTIST_NAME,
    fa.date_month AS RELEASE_MONTH,
    fa.release_count AS NUM_RELEASES
FROM (
    SELECT
        a.id AS artist_id,
        a.name AS artist_name,
        ri.date_month,
        COUNT(DISTINCT r.id) AS release_count
    FROM
        artist AS a
    JOIN
        artist_type AS at ON a.type = at.id
    JOIN
        artist_credit_name AS acn ON a.id = acn.artist
    JOIN
        release AS r ON r.artist_credit = acn.artist_credit
    JOIN
        release_info AS ri ON ri.release = r.id
    WHERE
        at.name = 'Person'
        AND a.name LIKE 'Elvis%'
        AND ri.date_month IS NOT NULL
    GROUP BY
        a.id, a.name, ri.date_month
) AS fa
```

```
WHERE
    fa.release_count = (
        SELECT
            MAX(sub.release_count)
        FROM (
            SELECT
                a2.id AS artist_id,
                COUNT(DISTINCT r2.id) AS release_count
            FROM
                artist AS a2
            JOIN
                artist_type AS at2 ON a2.type = at2.id
            JOIN
                artist_credit_name AS acn2 ON a2.id = acn2.artist
            JOIN
                release AS r2 ON r2.artist_credit = acn2.artist_credit
            JOIN
                release_info AS ri2 ON ri2.release = r2.id
            WHERE
                at2.name = 'Person'
                AND a2.name LIKE 'Elvis%'
                AND a2.id = fa.artist_id
                AND ri2.date_month IS NOT NULL
            GROUP BY
                a2.id, ri2.date_month
        ) AS sub
    )
    AND fa.date_month = (
        SELECT
            MIN(sub2.date_month)
        FROM (
            SELECT
                a3.id AS artist_id,
                ri3.date_month,
                COUNT(DISTINCT r3.id) AS release_count
            FROM
                artist AS a3
            JOIN
                artist_type AS at3 ON a3.type = at3.id
            JOIN
                artist_credit_name AS acn3 ON a3.id = acn3.artist
            JOIN
                release AS r3 ON r3.artist_credit = acn3.artist_credit
            JOIN
                release_info AS ri3 ON ri3.release = r3.id
            WHERE
                at3.name = 'Person'
```

```
                AND a3.name LIKE 'Elvis%'
                AND a3.id = fa.artist_id
                AND ri3.date_month IS NOT NULL
            GROUP BY
                a3.id, ri3.date_month
        ) AS sub2
        WHERE
            sub2.artist_id = fa.artist_id
            AND sub2.release_count = fa.release_count
    )
ORDER BY
    fa.release_count DESC,
    fa.artist_name ASC;
```

首先，查询通过内层的子查询计算每个符合条件的艺术家（类型为'Person' 且名字包含'Elvis'）在每个月的专辑发布数量，并确保每个发布的月份（'date_month'）非空。接下来，外层查询筛选出发布数量最多的月份，首先确定该艺术家发布最多专辑的最大数量，然后通过另一个子查询找出该最大发布数量对应的最早月份。最后，查询按艺术家的专辑发布数量降序排列，同时按艺术家名称升序排列，以确保结果按要求显示。查询最终输出的字段包括艺术家姓名、发布月份以及发布数量。

1.5.3 查询结果截图

	ARTIST_NAME	RELEASE_MONTH	NUM_RELEASES
1	Elvis Presley	1	56
2	Elvis Costello	9	19
3	Elvis Crespo	4	9
4	Elvis Depressedly	10	3
5	Elvis Perkins	2	2
6	Elvis Blue	9	1
7	Elvis Magno	2	1
8	Elvis Martínez	5	1
9	Elvis McFly	10	1
10	Elvis Nick	1	1

Figure 1.6 第五题查询结果

1.6 第六题

1.6.1 题目

列出 1900 年至 2023 年间每个十年在美国成立的团体数量。

具体要求：1900-2023 年间每十年在美国成立的类型为 Group 的艺术家的数量，对于每个十年使用如下所示的字符串展示：2000s。结果按十年的时间顺序进行排序。

1.6.2 sql 语句及分析

sql 语句：

```
select CAST(floor(a.begin_date_year / 10) * 10 AS TEXT) || 's' AS DECADE,
    count(*) as NUM_ARTIST_GROUP
from artist as a
join artist_type as at
on a.type = at.id
join area
on area.id = a.area
where at.name = 'Group'
and a.begin_date_year between 1900 and 2023
and area.name = 'United States'
group by floor(a.begin_date_year/10);
```

首先，查询通过对艺术家的 ‘begin_date_year‘ 字段进行除以 10 后取整操作，将艺术家所属的年代按十年为区间进行分类。通过 ‘CAST‘ 和 ‘||‘ 操作将年份转换为文本格式并添加”s” 后缀，生成以十年为单位的年代字符串（例如，”1970s”）。接着，查询通过 ‘JOIN‘ 操作将 ‘artist‘ 表、‘artist_type‘ 表和 ‘area‘ 表连接起来，确保仅统计艺术类型为”Group” 且地区为”United States” 的艺术家。同时，筛选出艺术家的 ‘begin_date_year‘ 在 1900 到 2023 年之间的记录。查询使用 ‘GROUP BY‘ 子句按年代进行分组，并计算每个年代内的艺术家数量（‘count(*)‘）。最终，查询按年代分组并输出每个年代的艺术家数量。

1.6.3 查询结果截图

1.7 第七题

1.7.1 题目

列出所有与匹兹堡交响乐团（Pittsburgh Symphony Orchestra）合作过的艺术家。

具体要求：按字母顺序打印艺术家姓名。如果艺术家姓名出现在表 artist credit 的同一条记录的 name 中，则该艺术家被视为合作者。艺术家姓名请始终使用表 artist 中的 name 字段。结果不包括匹兹堡交响乐团本身。

	DECADE	NUM_ARTIST_GROUP
1	1900s	6
2	1910s	15
3	1920s	63
4	1930s	69
5	1940s	102
6	1950s	356
7	1960s	1304
8	1970s	989
9	1980s	2215
10	1990s	4545
11	2000s	5788
12	2010s	2547
13	2020s	18

Figure 1.7 第六题查询结果

1.7.2 sql 语句及分析

sql 语句：

```
select distinct artist.name as ARTIST_NAME
from artist
join artist_credit_name on artist.id = artist_credit_name.artist
join artist_credit on artist_credit.id = artist_credit_name.artist_credit
where artist_credit.name in (select ac.name
                             from artist as a
                             join artist_credit_name as acn on a.id = acn.artist
                             join artist_credit as ac on ac.id = acn.artist_credit
                             where a.name = 'Pittsburgh Symphony Orchestra')
and artist.name is not 'Pittsburgh Symphony Orchestra'
order by artist.name asc;
```

查询首先通过子查询获取所有与”Pittsburgh Symphony Orchestra” 相关的艺术家信用名称，然后在主查询中使用这些名称来筛选出相应的艺术家姓名。主查询通过连接 ‘artist_credit_name‘ 和 ‘artist_credit‘ 表，确保返回的艺术家姓名与子查询中的名称匹配，并且排除了”Pittsburgh Symphony Orchestra” 的名字。最后，查询按艺术家姓名升序排列，确保结果中列出了所有相关艺术家的名字。

1.7.3 查询结果截图

	ARTIST_NAME
1	André Previn
2	Anthony Newman
3	Anton Bruckner
4	Antonio de Almeida
5	Antonín Dvořák
6	Boston Pops Orchestra
7	Béla Bartók
8	Camille Saint-Saëns
9	Children's Festival Chorus of Pittsburgh
10	Christian Sinding
11	David Zinman
12	Edward Elgar
13	Ferde Grofé
14	Franz Liszt
15	Franz Schubert
16	Fritz Reiner
17	George Frideric Handel
18	George Gershwin
19	Gregor Piatigorsky
20	Gustav Mahler
21	Henry Mazer
22	Horacio Gutiérrez
23	Isaac Stern

Figure 1.8 第七题查询结果

24	Itzhak Perlman
25	Jacques Offenbach
26	Jean Sibelius
27	Johann Sebastian Bach
28	Johannes Brahms
29	John Williams
30	Jonathan Leshnoff
31	Joseph Emil Villa
32	Julian Rachlin
33	Karl Goldmark
34	Klaus König
35	Leonidas Kavakos
36	Leoš Janáček
37	Leó Weiner
38	Lorin Maazel
39	Ludwig van Beethoven
40	Manfred Honeck
41	Marek Janowski
42	Mariss Jansons
43	Maurice Abravanel
44	Maurice Ravel
45	Mendelssohn Choir of Pittsburgh

Figure 1.9 第七题查询结果 (补)

46	Michael Rusinek
47	Michelle DeYoung
48	Misha Dichter
49	Nancy Goeres
50	Nathan Milstein
51	Ottorino Respighi
52	Pablo de Sarasate
53	Patricia Prattis Jennings
54	Peter Meven
55	Philharmonia Orchestra
56	Reid Neibaur Nibley
57	Richard Danielpour
58	Richard Strauss
59	Richard Wagner
60	Robert Bernat
61	Rolf Zuckowski
62	Rudolf Serkin
63	Samuel Barber
64	Sunhae Im
65	Susan Dunn
66	Utah Symphony

Figure 1.10 第七题查询结果 (补)

67	Victor Herbert
68	William Caballero
69	William Kapell
70	William Steinberg
71	Wolfgang Amadeus Mozart
72	Yan Pascal Tortelier
73	Yehudi Menuhin
74	Yo-Yo Ma
75	Zino Francescatti
76	Zoltán Kodály
77	Александр Константинович Глазунов
78	Дмитрий Борисович Кабалевский
79	Дмитрий Дмитриевич Шостакович
80	Михаил Иванович Глинка
81	Николай Андреевич Римский-Корсаков
82	Пётр Ильич Чайковский
83	Родион Константинович Щедрин
84	Сергей Васильевич Рахманинов
85	Сергей Сергеевич Прокофьев

Figure 1.11 第七题查询结果 (补)

1.8 第八题

1.8.1 题目

查找所有别名不包含 john 的 John。

具体要求：查找所有名字以 John 结尾的艺术家。列出他们拥有的别名数量以及通过逗号分隔的别名组成的字符串。别名字符串按字母顺序连接。排除别名包含单词 john（大写或小写）的艺术家，排除没有别名的艺术家。按艺术家姓名的字母顺序对结果进行排序。

1.8.2 sql 语句及分析

sql 语句：

```
select artist.name as ARTIST_NAME, count(artist_alias.id) as NUM_ALIASES,
    GROUP_CONCAT(artist_alias.name, ', ') as COMMA_SEPARATED_LIST_OF_ALIASES
from artist
join artist_alias on artist.id = artist_alias.artist
where artist.name like '%John'
and artist_alias.name is not null
and artist.id not in (select a.id
                        from artist as a
                        join artist_alias as aa on a.id = aa.artist
                        where aa.name like '%john%')
group by artist.name
order by artist.name;
```

查询首先通过 ‘artist‘ 表和 ‘artist_alias‘ 表的连接，筛选出艺术家名称包含”John” 且别名非空的记录。接着，使用 ‘GROUP_CONCAT‘ 函数将每个艺术家的所有别名按逗号分隔连接成一个字符串，并计算每个艺术家的别名数量 (‘NUM_ALIASES‘)。此外，查询通过子查询排除了那些别名中已包含”john”（不区分大小写）字符的艺术家。这样，查询的结果仅包括那些别名中没有”john” 字符的艺术家。最终，查询按照艺术家名称升序排列，并返回每个艺术家的姓名、别名数量以及按逗号分隔的别名列表。

1.8.3 查询结果截图

	ARTIST_NAME	NUM_ALIASES	COMMA_SEPARATED_LIST_OF_ALIASES
1	Anaïs St. John	1	Anaïs Brown
2	Andrw John	2	Juan Andres Gonzalez Alcantara, El Moncarca de la bachata
3	Barry St. John	1	Elizabeth Thompson
4	Fred St. John	1	Frederic L. Hostetler
5	Jon St. John	2	Duke Nukem, JSJ
6	Kate St John	1	ケイト・セント・ジョン
7	Mark St. John	1	Mark Leslie Norton
8	Mark and John	1	MARK AND JOHN
9	Persian John	1	P.J.

Figure 1.12 第八题查询结果

1.9 第九题

1.9.1 题目

比较 1950 年代和 2010 年代的音乐市场。找出市场份额增长最快的语言。

具体要求：列出对应语言在 1950 年代的发行专辑数量、在 2010 年代的发行专辑数量以及市场份额的差异。只列出市场份额增加的语言，并将结果四舍五入到最接近的千分之一。市场份额通过将该语言的发行数量除以相应十年的总发行数量来计算。不同地区的发行专辑按单独的发行专辑计算。

1.9.2 sql 语句及分析

sql 语句：

```
WITH DECADE_1950 AS (
    SELECT
        l.name AS LANGUAGE,
        COUNT(*) AS NUM_RELEASES
    FROM
        release
    INNER JOIN
        language l ON release.language = l.id
    INNER JOIN
        release_info ri ON release.id = ri.release
    WHERE
        ri.date_year BETWEEN 1950 AND 1959
    GROUP BY
        l.id
),

DECADE_2010 AS (
    SELECT
        l.name AS LANGUAGE,
        COUNT(*) AS NUM_RELEASES
    FROM
        release
    INNER JOIN
        language l ON release.language = l.id
    INNER JOIN
        release_info ri ON release.id = ri.release
    WHERE
        ri.date_year BETWEEN 2010 AND 2019
    GROUP BY
        l.id
),
```

```
T1 AS (
    SELECT
        COUNT(*)
    FROM
        release
    INNER JOIN
        release_info ri ON release.id = ri.release
    WHERE
        ri.date_year BETWEEN 1950 AND 1959
),

T2 AS (
    SELECT
        COUNT(*)
    FROM
        release
    INNER JOIN
        release_info ri ON release.id = ri.release
    WHERE
        ri.date_year BETWEEN 2010 AND 2019
)

SELECT
    DECADE_2010.LANGUAGE,
    COALESCE(DECADE_1950.NUM_RELEASES, 0) AS NUM_RELEASES_IN_1950s,
    COALESCE(DECADE_2010.NUM_RELEASES, 0) AS NUM_RELEASES_IN_2010s,
    ROUND(
        (
            CAST(COALESCE(DECADE_2010.NUM_RELEASES, 0) AS FLOAT) / (SELECT * FROM T2)
            - CAST(COALESCE(DECADE_1950.NUM_RELEASES, 0) AS FLOAT) / (SELECT * FROM T1)
        ),
        3
    ) AS INCREASE
FROM
    DECADE_2010
LEFT JOIN
    DECADE_1950 ON DECADE_1950.LANGUAGE = DECADE_2010.LANGUAGE
WHERE
    (
        CAST(COALESCE(DECADE_2010.NUM_RELEASES, 0) AS FLOAT) / (SELECT * FROM T2)
        - CAST(COALESCE(DECADE_1950.NUM_RELEASES, 0) AS FLOAT) / (SELECT * FROM T1)
    ) > 0
ORDER BY
    INCREASE DESC
LIMIT 1;
```

该 SQL 查询旨在比较 1950 年代和 2010 年代按语言分类的专辑发布数量，并计算出

这些语言在这两个时期之间的发布数量变化率（即增长率）。首先，查询通过两个子查询 'DECADE_1950' 和 'DECADE_2010' 分别计算出 1950 年代和 2010 年代每种语言的专辑数量。然后，通过 'T1' 和 'T2' 子查询计算出这两个时期的专辑总数，以便在计算增长率时使用。在最终的查询中，使用 'LEFT JOIN' 将这两个时期的语言专辑数据合并，并计算出每种语言在这两个时期的增长率。如果 2010 年代相比于 1950 年代的专辑发布数量有所增长（增长率大于 0），则将该语言及其增长率显示出来，结果按增长率降序排列，并限制显示增长率最高的语言。

1.9.3 查询结果截图

	LANGUAGE	NUM_RELEASES_IN_1950s	NUM_RELEASES_IN_2010s	INCREASE
1	Japanese	21	57240	0.032

Figure 1.13 第九题查询结果

1.10 第十题

1.10.1 题目

查找 1991 年出生的，其中恰好有四位歌手出现在 credit 中的男歌手的最新三张专辑。

具体要求：列出艺术家姓名、专辑名称和发行年份。按艺术家姓名排序，然后按发行日期（年、月、日）排序。排除没有发行年份的艺术家。对于给定的发行日期（年、月、日），每个专辑只输出 1 个条目。但是，如果专辑有多个发行日期，则应将它们视为不同的条目。

1.10.2 sql 语句及分析

sql 语句：

```
WITH DeduplicatedReleases AS (
    SELECT DISTINCT
        artist.name AS ARTIST_NAME,
        release.name AS RELEASE_NAME,
        release_info.date_year AS RELEASE_YEAR,
        release_info.date_month AS RELEASE_MONTH,
        release_info.date_day AS RELEASE_DAY
    FROM artist
    JOIN artist_credit_name ON artist.id = artist_credit_name.artist
    JOIN artist_credit ON artist_credit.id = artist_credit_name.artist_credit
    JOIN release ON release.artist_credit = artist_credit.id
    JOIN release_info ON release_info.release = release.id
    WHERE artist.gender = 1
      AND artist.begin_date_year = 1991
      AND release_info.date_year IS NOT NULL
      AND artist_credit.artist_count = 4
```

```
),
RankedReleases AS (
    SELECT
        ARTIST_NAME,
        RELEASE_NAME,
        RELEASE_YEAR,
        RELEASE_MONTH,
        RELEASE_DAY,
        ROW_NUMBER() OVER (
            PARTITION BY ARTIST_NAME
            ORDER BY RELEASE_YEAR DESC, RELEASE_MONTH DESC, RELEASE_DAY DESC
        ) AS row_num
    FROM DeduplicatedReleases
)
SELECT ARTIST_NAME, RELEASE_NAME, RELEASE_YEAR
FROM RankedReleases
WHERE row_num <= 3
ORDER BY ARTIST_NAME ASC, RELEASE_YEAR DESC, RELEASE_MONTH DESC, RELEASE_DAY DESC;
```

首先，通过 ‘DeduplicatedReleases‘ 子查询，查询了所有符合条件的艺术家和专辑数据，确保艺术家是男性且开始年份为 1991 年，且专辑的发布日期年份不为空，艺术家发布了四张专辑。接着，使用 ‘ROW_NUMBER()‘ 函数在 ‘RankedReleases‘ 子查询中为每位艺术家的专辑按发布日期进行排序，确保最新的专辑排在前面。最后，从结果中选取每位艺术家发布的最新三张专辑，按照艺术家名称升序排列，并按发布日期降序排序，输出艺术家的名称、专辑名称和发行年份。

1.10.3 查询结果截图

	ARTIST_NAME	RELEASE_NAME	RELEASE_YEAR
1	Akim	Pa' olvidarte (Panamá remix)	2019
2	Ale Mendoza	Está pa' mí (remix)	2018
3	Ale Mendoza	Piloteando la nave (remix)	2017
4	Alesso	Let Me Go	2017
5	Black Mayonnaise	Dick Goddard's Wintery Forest Revisited	2013
6	Burna Boy	Baddest	2015
7	B-Case	Feel It!	2013
8	Carnage	Hella Neck	2020
9	Chucho Flash	Hecha completa	2018
10	Deorro	Shakalaka	2018
11	Domo Genesis	Rella	2012
12	Dubzy	Hungry for Dis	2016
13	Ed Sheeran	Take Me Back to London (remix)	2019
14	Ed Sheeran	Lay It All on Me (Rudi VIP mix)	2015
15	Farruko	Celebration	2019
16	Farruko	Como soy II	2019
17	Farruko	Pa' olvidarte (remix)	2019
18	Frenna	Culo	2018
19	Ghostemane	Death by Dishonor	2017
20	Guru Randhawa	Saaho (Tamil) [Original Motion Picture Soundtrack]	2019
21	Guru Randhawa	Ban Ja Rani	2017
22	Jon Z	Bésame (remix)	2018
23	Jon Z	Pali2 (remix)	2018

Figure 1.14 第十题查询结果

24	Jon Z	Lo que yo diga (Dema Ga Ge Gi Go Gu remix)	2018
25	Kevin Flórez	Aquí se queda (remix)	2018
26	Kevin Flórez	Nuestra fiesta	2016
27	Lofty305	(RIP YAMS) SPACEGHOSTPUSSY	2016
28	London on da Track	Throw Fits	2019
29	London on da Track	Up Now	2018
30	London on da Track	Up Now	2018
31	Luan Santana	Com que roupa	2018
32	Matoma	Old Thing Back	2015
33	Mr Eazi	Tied Up	2018
34	Mr Eazi	Let Me Live (Banx & Ranx remix)	2018
35	Mr Eazi	Let Me Live	2018
36	PnB Rock	Leave Em Alone	2019
37	PnB Rock	Gang Up	2017
38	Quavo	100 Bands	2019
39	Quavo	100 Bands	2019
40	Quavo	No Brainer	2018
41	Simi Stylezz	Heaven or Hell	2019
42	Stephen	Crossfire, Part III	2017
43	Steven Malcolm	Oh My God (remix)	2019
44	Tyler, the Creator	Rella	2012

Figure 1.15 第十题查询结果 (补)

45	VI Seconds	Let Me in the Game	2013
46	Young Thug	Cheat Code Mode	2020
47	Young Thug	Bullets With Names	2020
48	Young Thug	Old Town Road (remix)	2019
49	Даниил Трифонов	Destination Rachmaninov: Arrival	2019
50	Даниил Трифонов	Destination Rachmaninov: Departure	2018
51	Даниил Трифонов	Preghiera / Piano Trios	2017
52	斉藤壮馬	SSSS.GRIDMAN CHARACTER SONG.1	2018
53	花江夏樹	欲張りDreamer	2018
54	花江夏樹	ヘヴィーオブジェクト キャラクターソング Vol.1	2016
55	花江夏樹	ミカグラ学園組曲 vol.2 キャラクターソングCD	2015
56	그냥노창	NO.MERCY (노머시) Part.5	2015
57	주영	NO.MERCY (노머시) Part.3	2015

Figure 1.16 第十题查询结果 (补)

第二章　第二部分

2.1 第一题

2.1.1 题目

In the instance of instructor show in the Figure 1, no two instructors have the same name. From this, can we conclude that name can be used as a superkey (or primary key) of instructor?

2.1.2 解答

超键（Superkey）：超键是指一个属性集合，能够唯一地标识表中的每一行。

主键（Primary Key）：主键是超键的一种特殊情况，它要求不仅能够唯一标识每一行，还必须是最小的（即不能包含多余的属性）。

姓名不能够作为唯一标识设为超键或主键。在实际应用中，数据库中很有可能出现同名讲师的情况，姓名就不再能够唯一标识每个讲师，从而导致数据库错误。因此，尽管在当前特定场景下除非数据库规定姓名不可重复，这样姓名才可以作为超键和主键，否则需要避免将姓名设为超键或主键，特别是在数据量增加或数据管理不严谨的情况下。在这种情况下，使用其他能够确保唯一性的属性（例如讲师 ID）作为主键会更加可靠。

2.2 第二题

2.2.1 题目

Suppose we have three relations r(A,B), s(B,C) and t(B,D), with all attributes declared as not null.

1) Give instances of relation r, s, and t such that in the result of (r natural left outer join s) natural left outer join t attribute C has a null value but attribute D has a non-null value.

2) Are there instances of r, s, and t such that the result of r natural left outer join (s natural left outer join t) has a null value for C but a non-null value for D? Explain why or why not.

2.2.2 解答

1) 例子如下：

表 1: 表 r 的数据示例 ($r(A, B)$)

A	B
1	10
2	20
3	30

表 2: 表 s 的数据示例 ($s(B, C)$)

B	C
10	100
30	300

表 3: 表 t 的数据示例 ($t(B, D)$)

B	D
10	500
20	600
30	700
40	800

表 4: 表格 r 和 s 的自然左外连接结果 (r natural left outer join s)

A	B	C
1	10	100
2	20	NULL
3	30	300

表 5: 表 r、表 s 和表 t 的自然左外连接结果 (r natural left outer join (s natural left outer join t))

A	B	C	D
1	10	100	500
2	20	NULL	600
3	30	300	700
NULL	40	NULL	800

2) 不存在。

因为是先进行 s 自然左外连接 t，连接后必然 C 没有空值，D 可能出现空值，再和 r 进行自然左外连接后，只可能出现 C 为空值 D 为空值或者 C 非空值 D 为空值，不可能出现 C 有空值 D 非空值。

2.3 第三题

2.3.1 题目

Given the following relation: Sailors <sid, name, age>, find the name and age of the oldest sailor(s), use “Orderby”clause in your query.

2.3.2 解答

sql 语句：

```
select Sailors.name, Sailors.age
from Sailors
where Sailors.age = (select sa.age
                     from Sailors as sa
                     order by age desc
                     limit 1)
order by Sailors.name;
```

该 SQL 语句的目的是查询所有年龄最大的水手的姓名和年龄，并按姓名字母顺序排列。首先，子查询 ‘select max(sa.age) from sailors as sa‘ 获取水手表中最大年龄的值，然后主查询通过 ‘where sailors.age = ...‘ 条件筛选出所有年龄等于最大值的水手。最后，使用 ‘order by sailors.name‘ 按水手的姓名进行升序排列，以确保查询结果按字母顺序展示。如果有多个水手的年龄相同且是最大值，他们将按名字排序后显示。

2.4 第四题

2.4.1 题目

Given the following relations:
Sailors <sid, name, age, ranking>
Reserves <sid, bid, day>
Boats <bid, name, color>
Find names of sailors who’ve reserved boat #103 and reserved only one time

2.4.2 解答

sql 语句：

```
select Sailors.name
from Sailors
where Sailors.sid in
        (select sa.sid
        from Sailors as sa
        join Reserves as re on sa.sid = re.sid
        join Boats on re.bid = Boats.bid
```

```
        where Boats.bid = 103
        group by sa.sid
        having count(sa.sid) = 1);
```

这段 SQL 语句首先通过子查询筛选出那些只预定过一次船编号为 ‘103‘ 的水手。首先，子查询将 ‘Sailors‘、‘Reserves‘ 和 ‘Boats‘ 表连接，过滤出预定船 ‘103‘ 且预定次数为 1 的水手（通过 ‘count(sa.sid) = 1‘ 实现），使用 ‘IN‘ 子句确保只返回那些在子查询结果中的水手。然后，主查询根据子查询返回的 ‘sid‘ 列表，从 ‘Sailors‘ 表中查找与这些 ‘sid‘ 匹配的水手姓名，最终返回符合条件的水手姓名。

2.5 第五题

2.5.1 题目

Given the following relations：

Dept <deptno, deptname, location>

Employees <id, name, deptno, salary>

List the deptno, deptname, and the max salary of all departments located in New York, use “Group By” clause in your query.

2.5.2 解答

sql 语句：

```
select max_salary_dept.deptno, max_salary_dept.deptname, max_salary_dept.max_salary
from (select Dept.deptno, Dept.deptname, Dept.location, max(Employees.salary) as max_salary
        from Employees as em
        right join Dept on em.deptno = Dept.deptno
        group by Dept.deptno) as max_salary_dept
where max_salary_dept.location = 'New York';
```

这段 SQL 查询的目的是从 `Employees` 和 `Dept` 表中找出所有位于”New York” 地区的部门的编号、部门名称以及该部门的最高薪资。首先，通过子查询计算每个部门的最高薪资，并且通过 `GROUP BY` 子句按部门编号进行分组，然后通过 right join 确保没有人的新部门也能得到对应的薪水（null）。然后，外部查询通过过滤条件 `WHERE max_salary_dept.location = 'New York'` 来只返回位于“New York”的部门，最终显示位于纽约的部门的编号、名称和最高薪资。